

Technology Stack

Date	04 July 2026
Team ID	
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	2 Marks

Define Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs used to build the solution. A well-chosen technology stack ensures that the application is scalable, maintainable, and aligned with the project requirements.

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side	HTML5, CSS3, Jinja2 templates (Flask)	Server-rendered pages keep the prediction form and result pages simple and fast to load, with no separate build step — well suited to a form-driven tool with a handful of pages.
2	Backend / Server-Side	Python, Flask	Flask is lightweight and integrates directly with the scikit-learn/XGBoost ecosystem, making it easy to load the trained model bundle and serve predictions with minimal boilerplate.
3	Database / Data Storage	Static files (Excel datasets + joblib model bundle)	With only 115 labelled training records and a model that only needs to be reloaded on retrain, flat files (.xlsx, floods.save) are sufficient — no live database is required.
4	Cloud / Hosting / Deployment	Docker, IBM Cloud Code Engine (Cloud Foundry / buildpack as alternative)	Containerizing the app with Docker keeps the environment consistent across machines, while the included Procfile and manifest.yml allow the same app to also deploy to buildpackbased PaaS.

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
5	Version Control & CI/CD	Git / GitHub	Standard distributed version control for tracking notebook, model, and app changes, and for collaborating on the project.
6	Third-Party APIs / Other Tools	scikit-learn, XGBoost, joblib, pandas, NumPy, Gunicorn	scikit-learn/XGBoost train and run the flood-prediction model; joblib serializes the model + scaler bundle; pandas/NumPy handle data prep; Gunicorn serves the app in production.